

Learn ESP32 for Home Automation

Lesson 1: Introduction to ESP32 and Development Environment

Introduction to ESP32 and Development Environment – Study Notes

Key Concepts

1. **ESP32 Architecture** – Dual core Tensilica LX6 MCU, Wi-Fi + Bluetooth, rich peripheral set.
2. **Arduino IDE Integration** – Adding the ESP32 board package via the Boards Manager.
3. **Toolchain & Drivers** – Required USB to UART drivers (CP210x, CH340, FTDI) and selecting the correct COM port.
4. **Flashing & Serial Monitoring** – Uploading sketches with the “Upload” button and using the Serial Monitor for debugging.
5. **“Blink” Sketch** – The classic “Hello World” for hardware: toggling an onboard LED to verify the toolchain works.

Summary

The ESP32 is a low cost, Wi-Fi/Bluetooth enabled microcontroller that can be programmed with the familiar Arduino IDE. To start, you must install the ESP32 board definitions by adding the appropriate URL to the IDE's *Additional Boards Manager URLs* field, then use the Boards Manager to download the package. After installing any needed USB UART drivers, select **“ESP32 Dev Module”** (or the exact board you have) and the correct COM port.

A simple “blink” program—`digitalWrite(LED_BUILTIN, HIGH/LOW)` inside `loop()`—confirms that the development environment is correctly set up and that the board can be programmed and communicate over serial. Once the sketch uploads successfully and the LED toggles, you're ready to explore more advanced ESP32 features such as Wi-Fi networking, sensor interfacing, and multitasking with FreeRTOS.

Important Points

- **Board URL:** `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json`
- **Arduino IDE version:** 1.8.19+ (or Arduino Pro IDE) – newer versions have built-in ESP32 support.
- **Common ESP32 boards:** ESP32 DevKitC, Wemos D1 /Mini /ESP32, ESP32 C3, ESP32 S2.
- **USB driver:** Install CP210x, CH340, or FTDI driver depending on your board's USB to UART chip.
- **Select board & port:** `Tools > Board > ESP32 Arduino > ESP32 Dev Module`; then `Tools > Port > COMx`.
- **Flash size & partition scheme:** Default “Default 4MB with 1.5MB SPIFFS” works for most beginner projects.

- **Blink sketch uses `LED\_BUILTIN`** (GPIO /2 on most dev kits) – may differ; consult your board's schematic.
- **Serial monitor baud rate:** 115200 (default for ESP32) – set the same in `Serial.begin(115200);``.

Examples

1. Basic Blink Sketch

```
```cpp
// Blink – verifies ESP32 setup
void setup() {
 pinMode(LED_BUILTIN, OUTPUT); // LED on GPIO2 (most dev boards)
 Serial.begin(115200); // optional: for debug output
 Serial.println("\n--- ESP32 Blink started ---");
}
void loop() {
 digitalWrite(LED_BUILTIN, HIGH); // LED on
 delay(500); // wait 500 /ms
 digitalWrite(LED_BUILTIN, LOW); // LED off
 delay(500); // wait 500 /ms
}
```
```

***Explanation:** `setup()` runs once, configuring the LED pin and starting the serial console. `loop()` repeatedly turns the LED on/off with a 500 /ms pause, creating a visible blink.

2. Blink with Serial Feedback

```
```cpp
int ledState = LOW;
unsigned long previousMillis = 0;
const long interval = 1000; // 1 second
void setup() {
 pinMode(LED_BUILTIN, OUTPUT);
 Serial.begin(115200);
 Serial.println("Blink with timer");
}
void loop() {
 unsigned long current = millis();
 if (current - previousMillis >= interval) {
 previousMillis = current;
 ledState = !ledState; // toggle state
 digitalWrite(LED_BUILTIN, ledState);
 Serial.print("LED is now ");
 Serial.println(ledState ? "ON" : "OFF");
 }
}
```
```

```
}  
...
```

Explanation: Uses a non blocking timer (`millis()`) instead of `delay()`. This pattern is essential when you later add Wi-Fi or sensor code that must run concurrently.

Quick Review

1. **What URL must be added to the Arduino IDE to install ESP32 board definitions?**
2. **Which menu path selects the ESP32 board you are using?**
3. **What is the default GPIO for the built-in LED on most ESP32 dev kits?**
4. **Why might you prefer `millis()` over `delay()` in a sketch?**
5. **Name two USB-UART driver chips you might need to install for ESP32 boards.**

Further Reading

- **ESP32 Architecture Deep Dive** – CPU cores, memory map, peripherals.
- **Wi-Fi Programming with Arduino Core** – `WiFi.h`, connecting to networks, HTTP client/server.
- **Bluetooth Low Energy (BLE) on ESP32** – `BLEDevice.h`, creating services/characteristics.
- **FreeRTOS on ESP32** – tasks, queues, and synchronization primitives.
- **ESP-NOW and Mesh Networking** – peer-to-peer communication without Wi-Fi AP.

Happy hacking! Ø=Þ€

